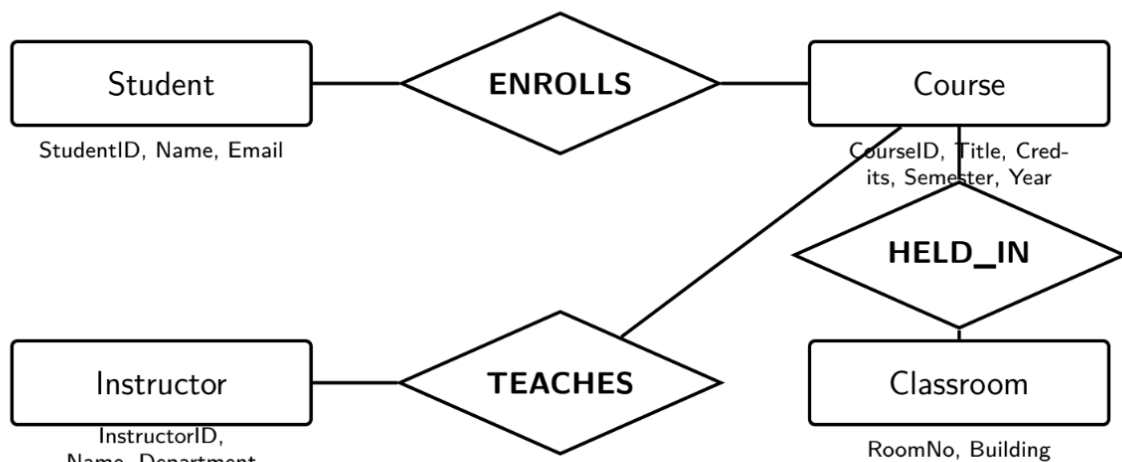


DBMS Lab Assignment 1

Boddupalli Jahnavi
24MAB0A07

Question 1 — University Course Registration



Study the diagram first. Business rules are on the next slide.

Question 1 — Business Rules & Task

Business rules

- ▶ The same course is offered in many semesters and years.
- ▶ A course may have multiple sections in the same semester.
- ▶ Different sections may be taught by different instructors.
- ▶ A student registers for a particular section, not merely for a course.
- ▶ Classroom allocation may differ across sections.

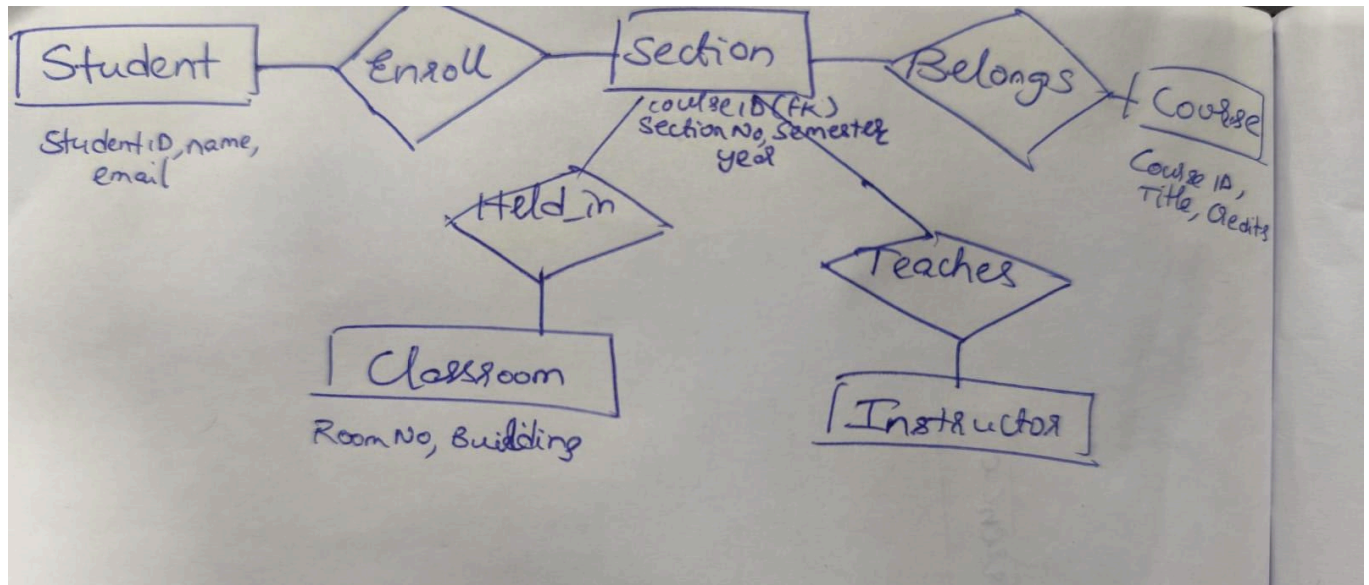
Find exactly 4 issues.

For each issue:

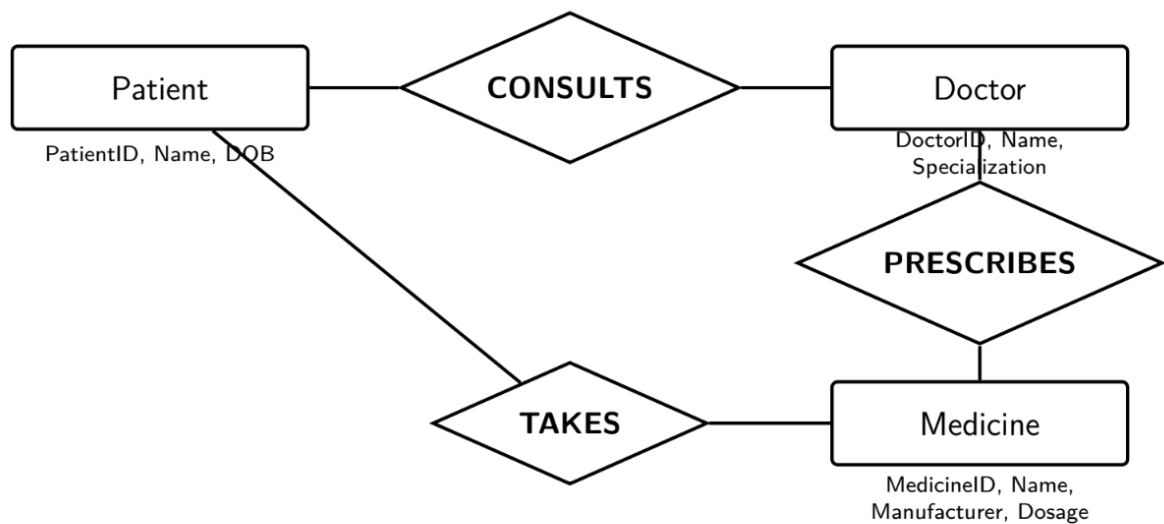
1. Identify the flaw.
2. Explain the information loss.
3. Give the minimal ER correction.

Flaws:

- 1) The ENROLLS relationship links Student directly to Course. This means we can't tell which specific offering semester, year, or section, a student actually registered for. The fix is to introduce a weak entity Section(SectionNo, Semester, Year), identified by CourseID, and change ENROLLS to run between Student and Section instead of Student and Course.
- 2) TEACHES links Instructor directly to Course. Since different sections of the same course, in the same semester, can be taught by different instructors, tying the relationship to Course forces every section to wrongly share one instructor. The fix is to re-point TEACHES from Instructor–Course to Instructor–Section.
- 3) HELD_IN links Course directly to Classroom, which makes room allocation a fixed property of the course itself. But different sections meet in different rooms, so this can't be represented. The fix is to re-point HELD_IN from Course–Classroom to Section–Classroom.
- 4) Semester and Year are stored as single-valued attributes of the Course entity. Since a course is offered repeatedly across many semesters and years, a single-valued attribute can only hold one semester/year per course row — so either history of repeated offerings is lost, or you're forced to duplicate Course rows, which breaks entity integrity. The fix is to remove Semester/Year from Course and move them onto the new Section entity, one row per offering.



Question 2 — Hospital Prescription System



Study the diagram first. Business rules are on the next slide.

Question 2 — Business Rules & Task

Business rules

- ▶ A patient may consult the same doctor many times.
- ▶ Different consultations may result in different prescriptions.
- ▶ The dosage of the same medicine may differ by patient and visit.
- ▶ The same medicine may be prescribed again with a different frequency or duration.
- ▶ A prescription must identify **who prescribed what, to whom, and when**.

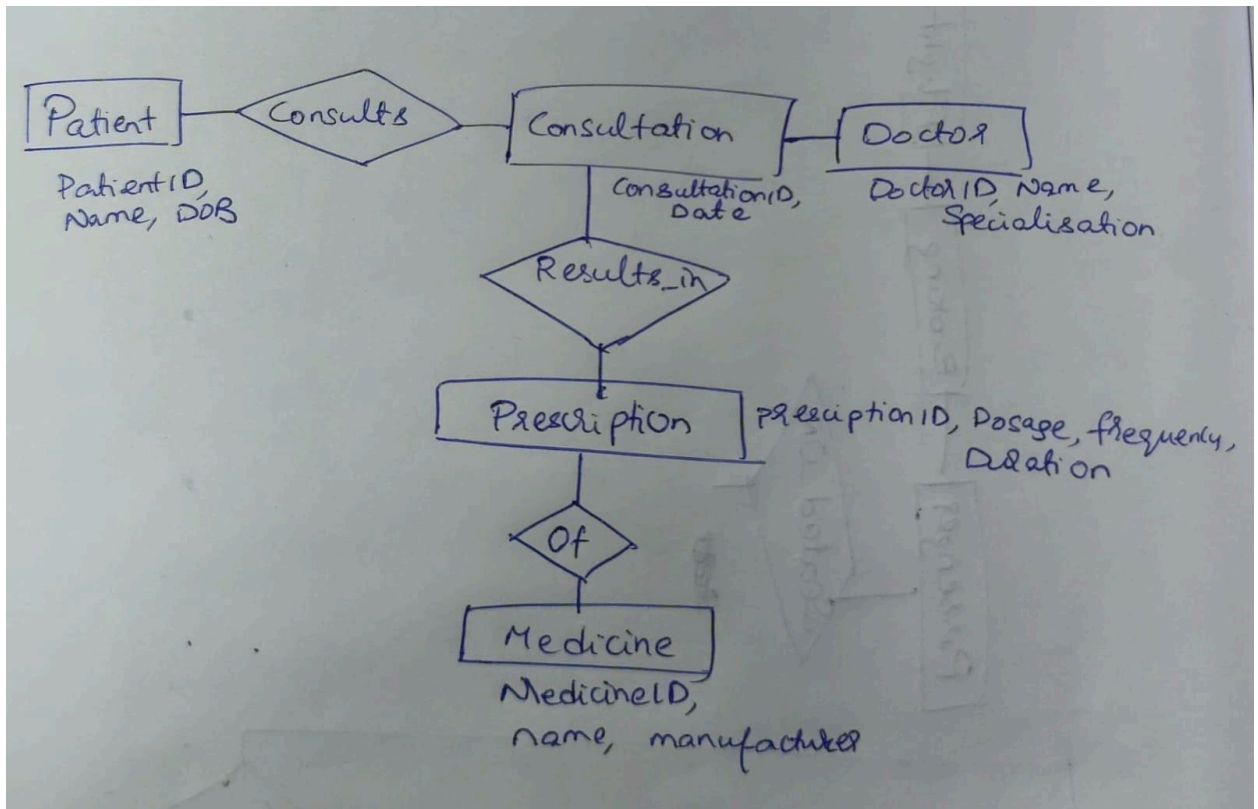
Find exactly 4 issues.

Your redesign must preserve:

- repeated consultations,
- prescription history,
- patient-specific dosage,
- time-dependent instructions.

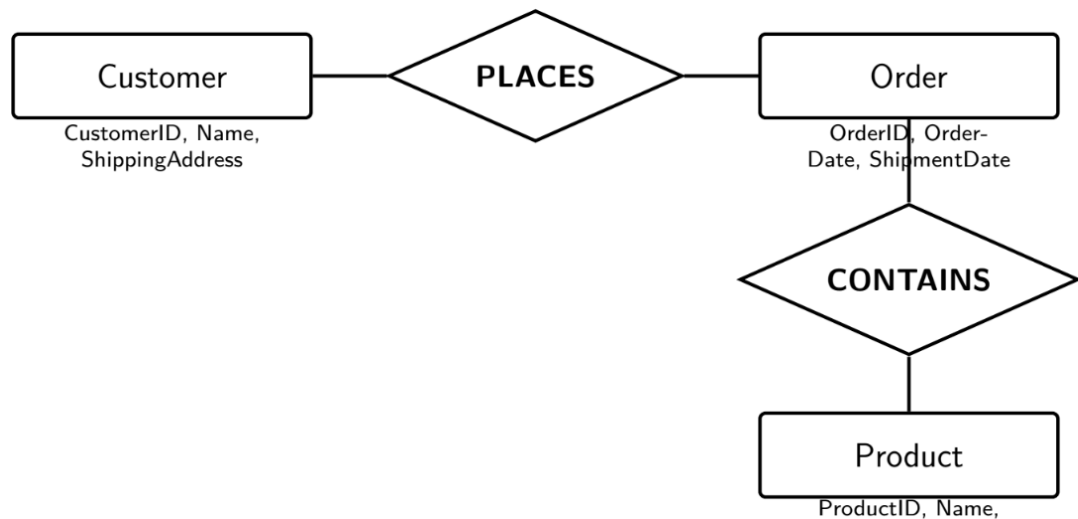
Flaws:

- 1) CONSULTS is a plain Patient–Doctor relationship with no date or visit attribute, and no Consultation entity. So repeated consultations between the same patient and doctor can't be told apart. The fix is to introduce a weak entity Consultation(ConsultationID, Date), identified by (PatientID, DoctorID), so CONSULTS runs through Consultation.
- 2) PRESCRIBES links Doctor directly to Medicine and bypasses the Patient entirely. That violates the rule that a prescription must identify who prescribed what, to whom, and when there's no way to state which patient a medicine was prescribed to, or during which visit. The fix is to replace PRESCRIBES with a Prescription entity attached to Consultation (so doctor, patient, and date are all implied) and linked onward to Medicine.
- 3) Dosage is a single-valued attribute of the Medicine entity. But dosage varies by patient and by visit, so this design can't record two different dosages of the same medicine for two different patients. The fix is to remove Dosage from Medicine and make it an attribute of the Prescription entity instead.
- 4) TAKES is a direct Patient-Medicine relationship with no way to distinguish repeat prescriptions of the same medicine. The fix is to drop TAKES entirely, each prescribing event becomes its own Prescription row with its own PrescriptionID, so the same medicine can appear multiple times for a patient, each with its own Dosage, Frequency, and Duration.



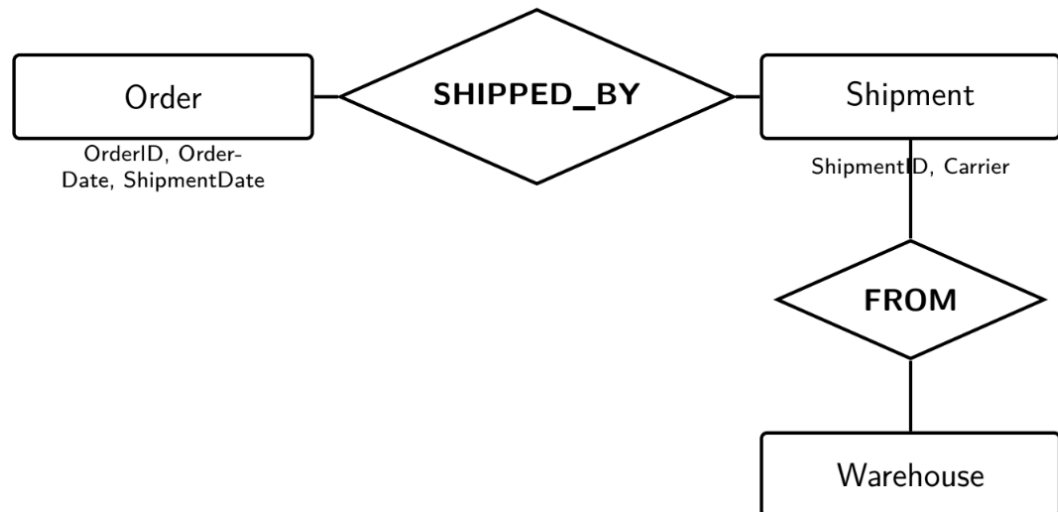
Question 3 — E-Commerce Order Fulfilment

Flawed ER Design — Part A



Question 3 — E-Commerce Order Fulfilment

Flawed ER Design — Part B



Question 3 — Business Rules & Task

Business rules

- ▶ A customer may use a different delivery address for every order.
- ▶ Quantity is specific to an order line, not to the product itself.
- ▶ One order may be split into several shipments.
- ▶ A single order line may be partially shipped from different warehouses.
- ▶ Each shipment has its own dispatch date and carrier information.

Find exactly 4 issues.

The corrected model must support:

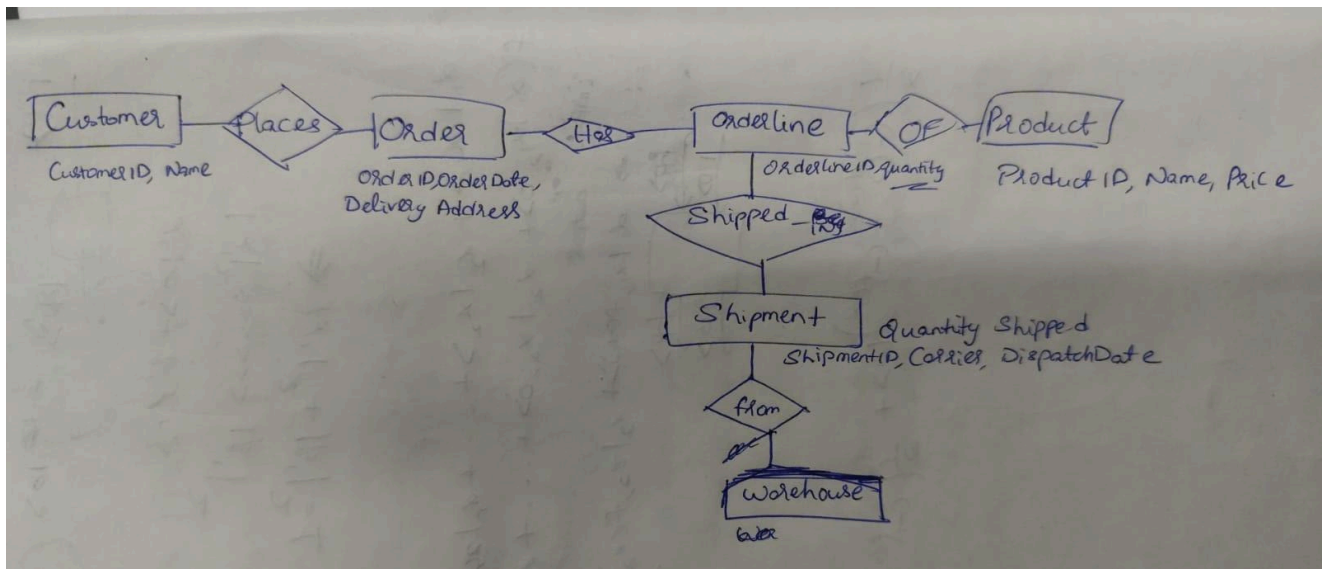
- historical order data,
- line-item quantity,
- split shipments,
- partial fulfilment.

Flaws:

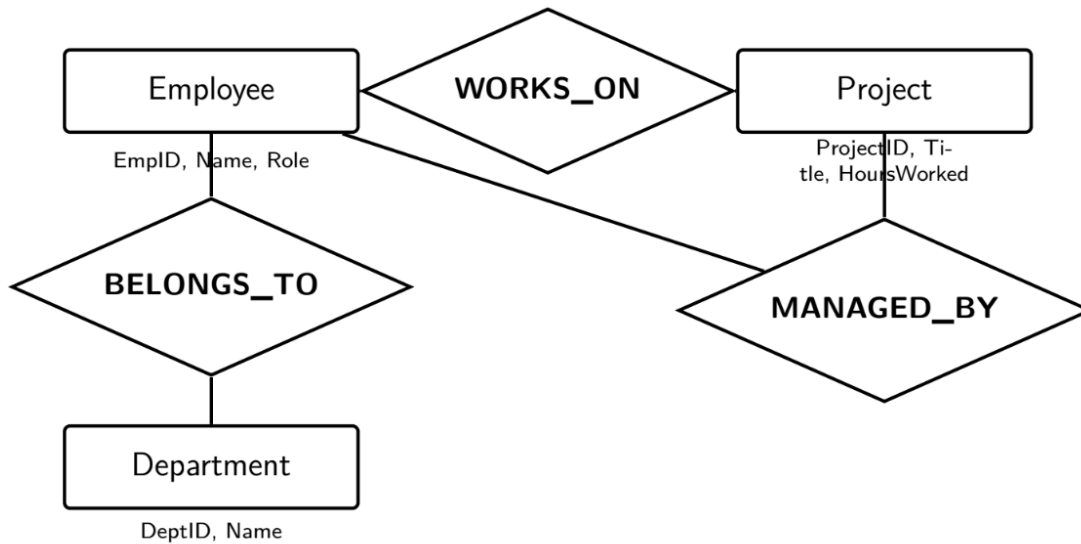
- 1) ShippingAddress is a single-valued attribute of the Customer entity, which binds every order to one fixed address per customer. But a customer may use a different delivery address for every order. The fix is to remove ShippingAddress from Customer and add it as an attribute of Order instead.
- 2) Quantity is a single-valued attribute of the Product entity. That makes it impossible to record different quantities of the same product across different orders. The fix is to

convert CONTAINS into an associative entity OrderLine(OrderLineID, Quantity) between Order and Product, and move Quantity there.

- 3) ShipmentDate is a single-valued attribute of Order, which assumes one order has exactly one shipment and one date. But an order may be split into several shipments dispatched on different dates, and this design can't hold more than one date per order. The fix is to remove ShipmentDate from Order and keep dispatch date as an attribute of the Shipment entity, so each shipment carries its own date.
- 4) SHIPPED_BY connects Order directly to Shipment at order-level and FROM connects Shipment directly to Warehouse. That means a single order line being partially shipped from multiple warehouses can't be represented — shipment is tied to the whole order, not to individual line items. The fix is to re-point shipment linkage to line-item by relating Shipment to OrderLine (with a QuantityShipped attribute) instead of to Order directly, so one order line can map to several (Shipment, Warehouse) pairs.



Question 4 — Project Staffing and Roles



Question 4 — Business Rules & Task

Business rules

- ▶ An employee may join, leave, and later rejoin the same project.
- ▶ The employee's role may be different on different projects.
- ▶ Hours are recorded per employee-project assignment and time period.
- ▶ Project managers may change during the lifetime of a project.
- ▶ Management history must be retained for audit purposes.

Find exactly 4 issues.

Focus on information that is:

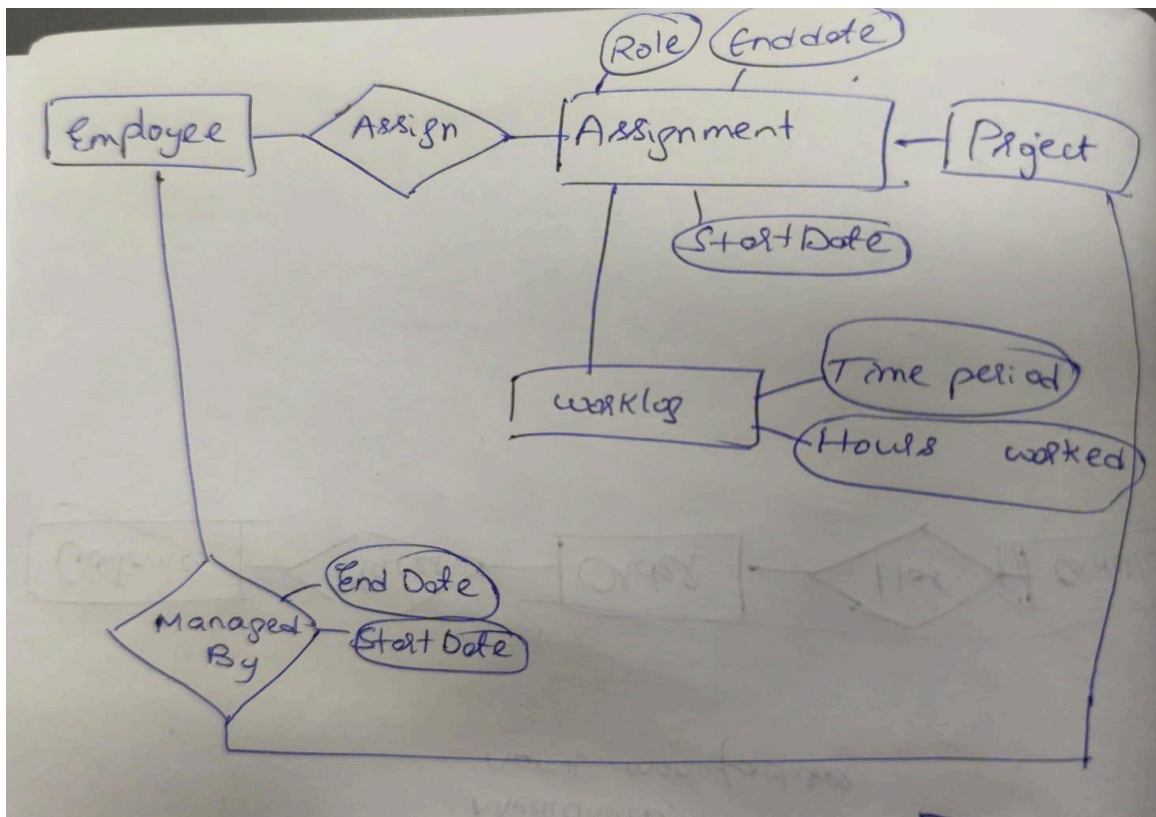
- assignment-specific,
- time-dependent, or
- historical.

Flaws:

- 1) There should be an Assignment entity which stores the role of that particular employee in a particular project. The role cannot be stored as an attribute of Employee because the same employee may have different roles in different projects. Therefore, Role should be stored in the Assignment entity because it depends on the particular Employee-Project assignment.
- 2) The WORKS_ON relationship cannot represent an employee joining, leaving, and rejoining the same project. A simple Employee-Project relationship can only indicate that the

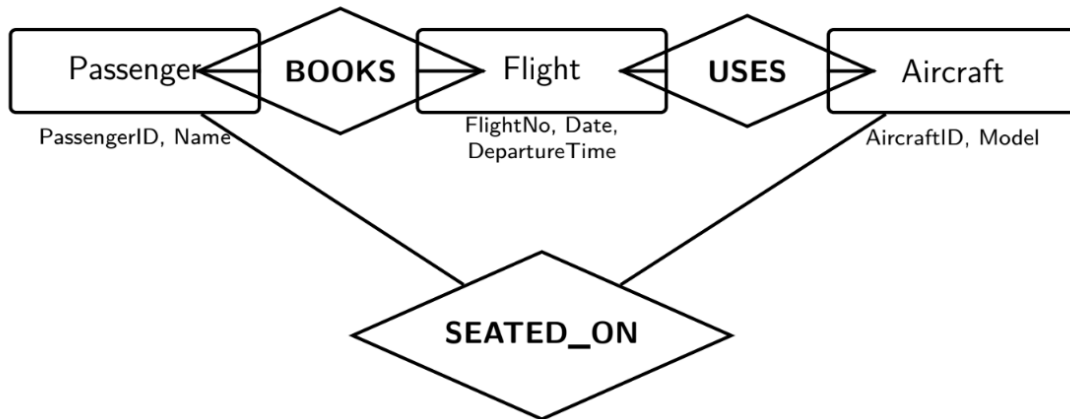
employee works on the project and cannot preserve the separate periods. Therefore, the Assignment entity should contain StartDate and EndDate attributes. Each time the employee joins the project, a separate Assignment record can be created, allowing the complete Employee–Project history to be maintained

- 3) Since hours are recorded per employee-project assignment and per time period, HoursWorked cannot simply be stored in the Project entity. It also should not be stored directly in Assignment because an employee may work different numbers of hours during different time periods within the same assignment. Therefore, a separate WorkLogs entity should be created. WorkLogs should store the hours worked for a particular Assignment during a particular time period, such as a week or month.
- 4) The MANAGED relationship between Employee and Project should also retain StartDate and EndDate attributes because project managers can change over time. If we store only the current manager, the previous manager and the period during which they managed the project would be lost. Therefore, the MANAGED relationship should have StartDate and EndDate attributes so that the complete management history can be retained for audit purposes.



Question 5 — Airline Booking and Seat Assignment

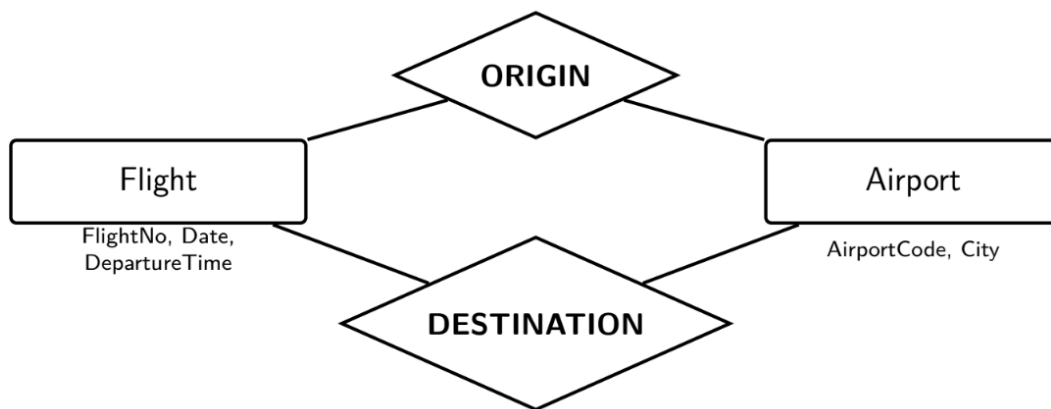
Flawed ER Design — Part A



Part B of the same ER model is on the next slide.

Question 5 — Airline Booking and Seat Assignment

Flawed ER Design — Part B



Parts A and B together form **one flawed ER model**.

Question 5 — Business Rules & Task

Business rules

- ▶ A flight number operates on many dates.
- ▶ One PNR may contain several passengers and several flight segments.
- ▶ Aircraft assignment may change for one particular flight occurrence.
- ▶ A seat is assigned to a passenger for a specific flight occurrence.
- ▶ The same passenger may have different seats on different segments.

Find exactly 4 issues.

The corrected model must represent:

- PNR/bookings,
- flight occurrences,
- aircraft changes,
- segment-specific seats.

Flaws:

- 1) BOOKS is a plain, direct Passenger–Flight relationship, with no Booking/PNR entity at all. This means several passengers and several flight segments can't be grouped under one reservation reference every passenger-flight pair sits as an isolated fact instead of parts of one PNR. The fix is to introduce a Booking(PNR) entity, with Passenger–Booking and Booking–Flight relationships, so one Booking can reference many passengers and many flight segments.
- 2) SEATED_ON links Passenger directly to Aircraft rather than to a specific flight occurrence. That ties a seat to the aircraft itself rather than to one flight segment. The fix is to re-point SEATED_ON from Passenger–Aircraft to Passenger–Flight (i.e., a booking-line to a flight occurrence), with SeatNo as an attribute of that relationship.
- 3) Flight bundles FlightNo, a recurring schedule identity together with Date and DepartureTime as if they're just attributes of one undifferentiated entity. Since a flight number operates on many dates, this blurs schedule-level data from occurrence-level data and leaves it structurally unclear how many rows exist per flight number. The fix is to make FlightNo the identifying attribute of a Flight (schedule) entity, and treat (FlightNo, Date) as a weak Flight-Occurrence entity under it, so each date is a distinct, addressable occurrence.
- 4) USES (Flight–Aircraft) is a plain relationship holding only the current aircraft for a flight occurrence. The fix is to convert USES into an associative entity AircraftAssignment(AssignedOn) between FlightOccurrence and Aircraft, so each (re)assignment is its own time-stamped row.

